

Accessing Remote Data

CS 111

Fall 2024

Operating System Principles

Peter Reiher

Outline

- Data on other machines
- Remote file access architectures
- Challenges in remote data access
 - Security
 - Reliability and availability
 - Performance
 - Scalability

Remote Data: Goals and Challenges

- Sometimes the data we want isn't on our machine
 - A file
 - A database
 - A web page
- We'd like to be able to access it, anyway
- How do we provide access to remote data?

Basic Goals

- Transparency
 - Indistinguishable from local files for all uses
 - All clients see all files from anywhere
- Performance
 - Per-client: at least as fast as local disk
 - Scalability: unaffected by the number of clients
- Cost
 - Capital: less than local (per client) disk storage
 - Operational: zero, it requires no administration
- Capacity: unlimited, it is never full
- Availability: 100%, no failures or service down-time

Key Characteristics of Remote Data Access Solutions

- APIs and transparency
 - How do users and processes access remote data?
 - How closely does remote data resemble local data?
- Performance, robustness, and synchronization
 - Is remote data as fast and reliable as local data?
 - Is synchronized access to remote data like local?
- Architecture
 - How is the solution integrated into clients and servers?
- Protocol and work partitioning
 - How do client and server cooperate?

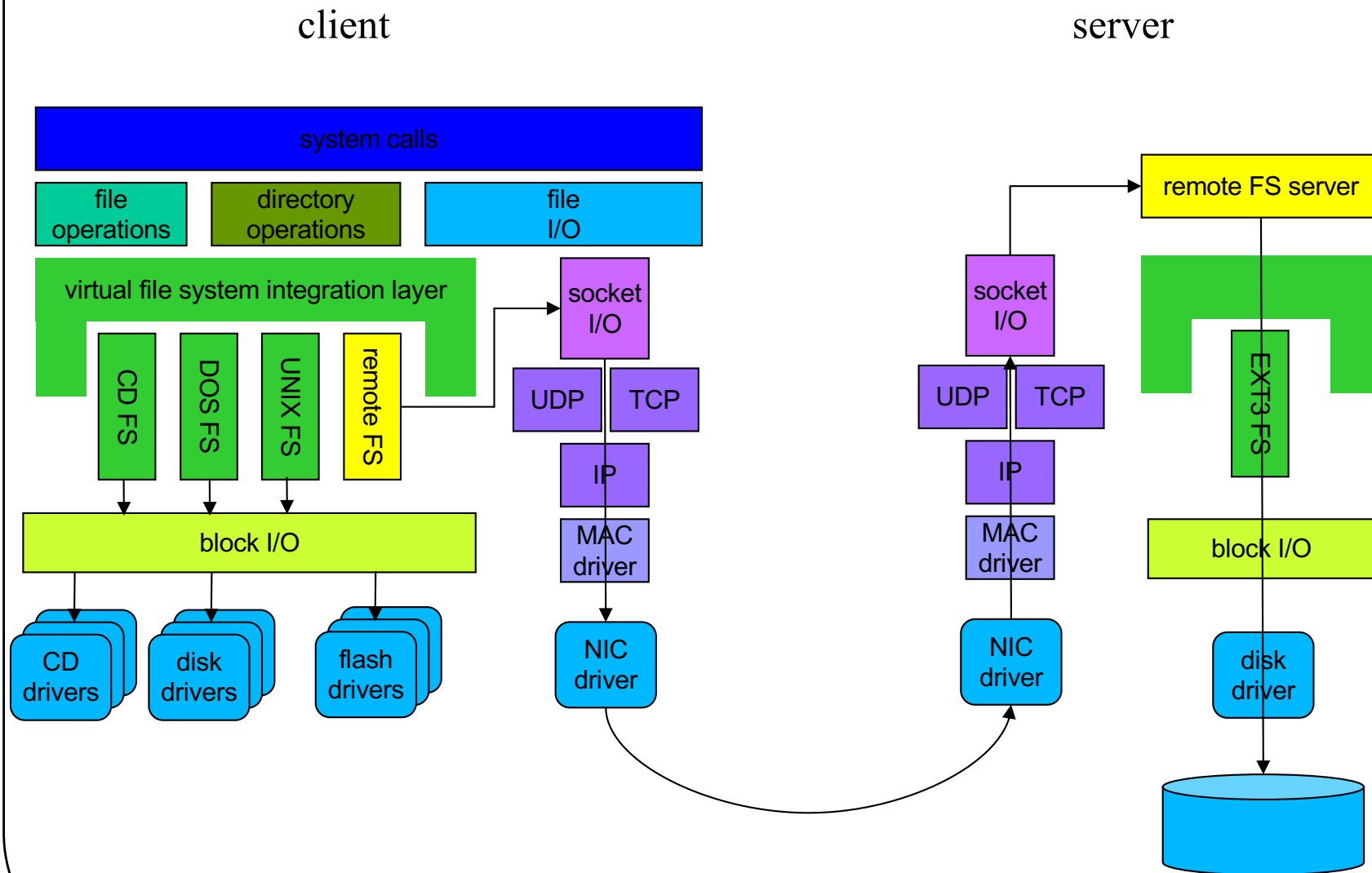
Remote File Access Architectures

- Remote file transfer
- Remote disk access
- **Remote file access**
- Distributed file system
- Cloud model

Remote File Access

- Goal: complete transparency
 - Normal file system calls work on remote files
 - Support file sharing by multiple clients
 - Performance, availability, reliability, scalability
- Typical architecture
 - Exploits plug-in file system architecture
 - Client-side file system is a local proxy
 - Translates file operations into network requests
 - Server-side daemon receives/process requests
 - Translates them into real file system operations

Remote File Access Architecture



For Example,

- A remote file system client is paired with a remote file system server on another machine
- Access to remote files goes through the file system interface and VFS to the local client
- That client formats messages to be sent to the remote server
 - Which are transmitted through the network I/O protocol stack
- The messages transit the network to the remote machine hosting the server

Completing the Example

- The remote file system server translates the messages into local file system ops
 - E.g., fetching a block of data to satisfy a read request
- The data is packaged into network messages
- Which are sent back to the client's machine
- That machine's client code provides the results to the requesting process

Remote File Access: Pros and Cons

- Advantages
 - Very good application level transparency
 - Very good functional encapsulation
 - Able to support multi-client file sharing
 - Potential for good performance and robustness
- Disadvantages
 - At least part of implementation must be in the OS
 - Client and server sides tend to be fairly complex
- This is THE model for client/server storage

Security For Remote File Systems

- Major issues:
 - Privacy and integrity for data on the network
 - Solution: encrypt all data sent over network
 - Authentication of remote users
 - Solution: various approaches
 - Trustworthiness of remote sites
 - Solution: various approaches

Authentication Approaches

- Anonymous access
- Peer-to-peer approaches
- Server authentication approaches
- Domain authentication approaches

Peer-to-Peer Authentication

- All participating nodes are trusted peers
- Client-side authentication/authorization
 - All users are known to all systems
 - All systems are trusted to enforce access control
 - Example: basic NFS
- Advantages:
 - Simple implementation
- Disadvantages:
 - You can't always trust all remote machines
 - Doesn't work in heterogeneous OS environment
 - Universal user registry is not scalable

Server Authenticated Approaches

- Client agent authenticates to each server
 - Authentication used for entire session
 - Authorization based on credentials produced by server
 - Example: Login-based FTP, SCP, CIFS
- Advantages
 - Simple implementation
- Disadvantages
 - May not work in heterogeneous OS environment
 - Universal user registry is not scalable
 - No automatic fail-over if server dies

Domain Authentication Approaches

- Independent authentication of client & server
 - Each authenticates with independent authentication service
 - Each knows/trusts only the authentication service
- Authentication service may issue signed “tickets”
 - Assuring each of the others’ identity and rights
 - May be revocable or timed lease
- May establish secure two-way session
 - Privacy – nobody else can snoop on conversation
 - Integrity – nobody can generate fake messages
- Kerberos is one example

Distributed Authorization

1. Authentication service returns credentials
 - Which server checks against Access Control List
 - Advantage: auth service doesn't know about ACLs
2. Authentication service returns capabilities
 - Which server can verify (by signature)
 - Advantage: servers do not know about clients
- Both approaches are commonly used
 - Credentials: if subsequent authorization required
 - Capabilities: if access can be granted all-at-once

Trustworthiness of Remote Sites

- Sometimes based on administrative issues
 - E.g., all sites controlled by one company/administrator
- Some authentication solutions limit need for trust
 - E.g., Kerberos
- Cryptographic approaches sometimes possible
 - E.g., if you don't trust your cloud storage site, only store encrypted data there

Reliability and Availability

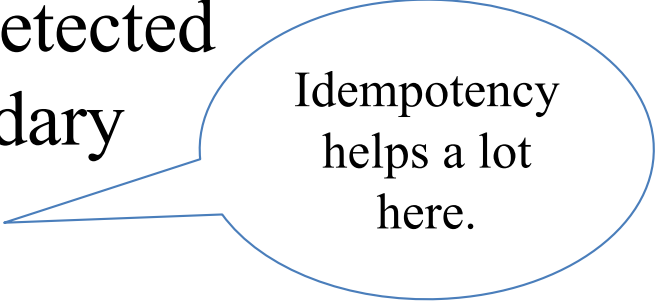
- *Reliability* is high degree of assurance that service works properly
 - Challenging in distributed systems, because of partial failures
 - Data should not be lost despite failures
- *Availability* is high degree of assurance that service is available whenever needed
 - Failures of some system elements don't prevent data access
 - Certain kinds of distributed systems can greatly improve availability
- Both, here, in the context of accessing remote files

Achieving Reliability

- Must reduce probability of data loss
- Typically by some form of redundancy
 - Disk/server failures must not result in data loss
 - Spread data across multiple disks, with redundancy
 - Copies on multiple servers
 - Backup, at the worst
- Also important to automatically recover after failure
 - Remote copies of data become available again
 - Redundancy loss due to failure must be made up

Availability and Fail-Over

- Fail-over means transferring work/requests from failed server to some other server
- Data must be mirrored to secondary server
- Failure of primary server must be detected
- Client must be failed-over to secondary
- Session state must be reestablished
 - Client authentication/credentials
 - Session parameters (e.g., working directory, offset)
- In-progress operations must be retransmitted
 - Client must expect timeouts, retransmit requests
 - Client responsible for writes until server ACKs



Idempotency
helps a lot
here.

Availability: Failure Detect/Rebind

- If a server fails, clients need to detect it and rebind to a different server
- Client driven recovery
 - Client detects server failure (connection error)
 - Client reconnects to (successor) server
 - Client reestablishes session
- Transparent failure recovery
 - System detects server failure (health monitoring)
 - Successor assumes primary's IP address (or other redirection)
 - State reestablishment
 - Successor recovers last primary state check-point
 - Stateless protocol

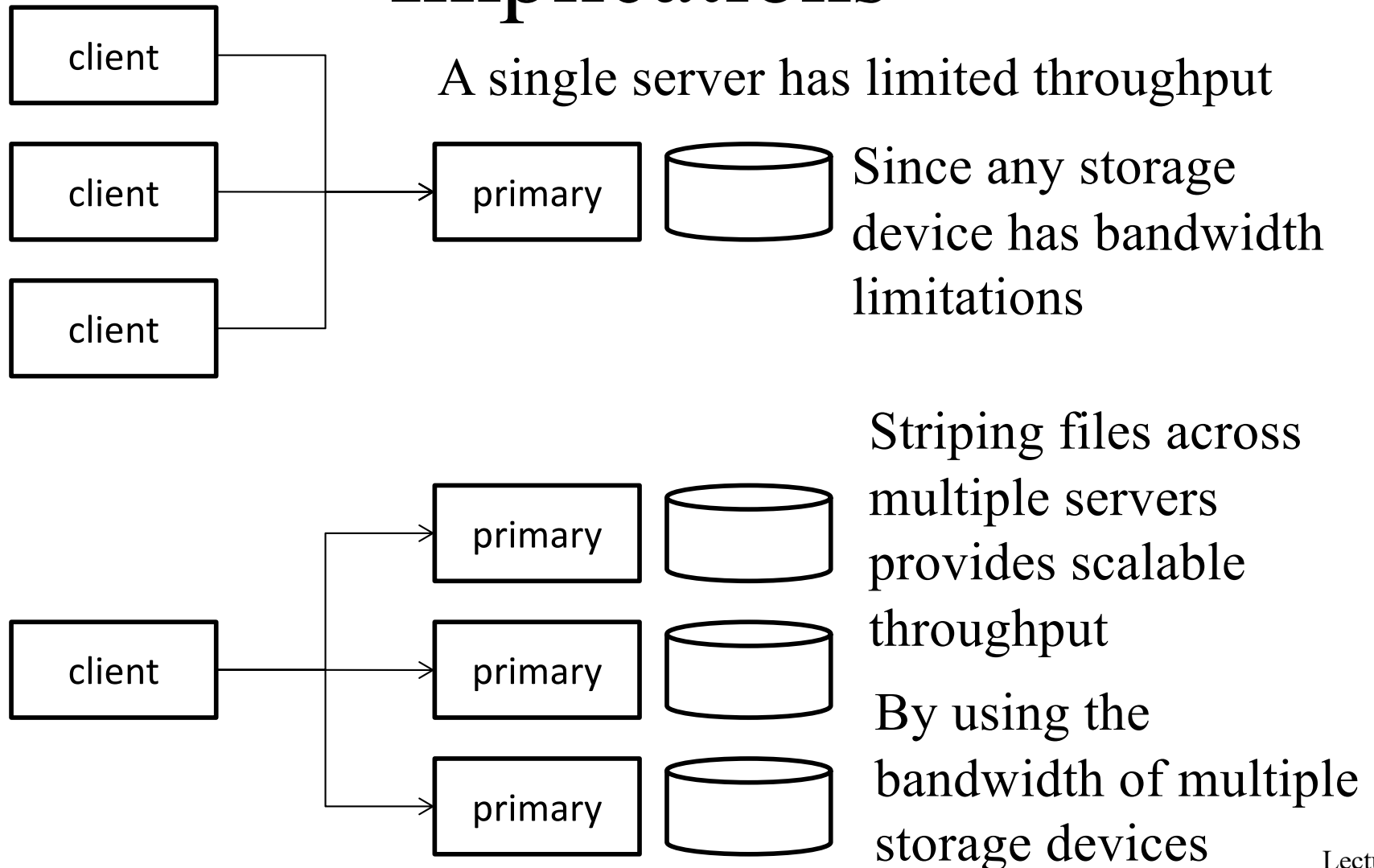
Availability: Stateless Protocols

- Stateful protocols (e.g., TCP)
 - Operations occur within a context
 - Server must save state to maintain that context
 - Each operation depends on previous operations
 - Replacement server must obtain session state to operate properly
- Stateless protocols (e.g., HTTP)
 - Client supplies necessary context with each request
 - Each operation is self-contained and unambiguous
 - Successor server needs no memory of past events
- Stateless protocols make fail-over easy

Remote File System Performance

- Storage device bandwidth and performance
- Performance for reads
- Performance for writes
- Overheads particular to remote file systems

Storage Device Bandwidth Implications

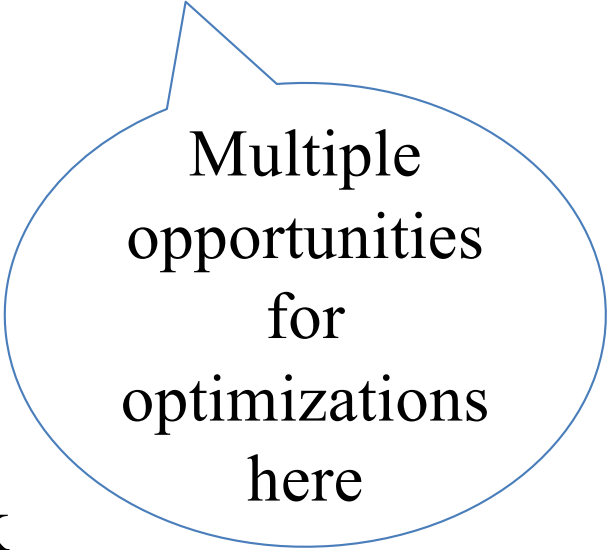


Network Impacts on Performance

- Bandwidth limitations
 - Implications for client
 - Implications for server
- Delay implications
 - Particularly important if acknowledgements required
- Packet loss implications
 - If loss rate high, will require acknowledgements

Performance of Reads

- Most file system operations are reads, so read performance is critical
- Network read consists of several steps:
 - Client application requests read
 - Read request is sent via network
 - Server receives read request
 - Server fetches requested data
 - Server sends data across network
 - Client receives data and gives it to application



Multiple
opportunities
for
optimizations
here

Read Caching in Remote File Systems

- Common way to improve read performance is through caching
- Can use read-ahead, but costs of being wrong are higher than for local disk
 - Though benefits are also higher
 - Client can speculatively request reads
 - Or server can speculatively perform them
 - And either send them or cache them locally

Caching For Reads

You can
do both!

- Client-side caching
 - Data permanently stored at the server is cached at the client
 - Eliminates waits for remote read requests
 - Reduces network traffic
 - Reduces per-client load on server
- Server-side caching
 - Typically performed similarly to single machine caching
 - Reduces disk delays, but not network costs

But potential consistency issues

Whole File Vs. Block Caching

- Many distributed file systems use whole file caching
 - E.g., AFS
- Higher network latency justifies whole file pulls
- Stored in local (cache-only) file system
- Satisfy early reads before entire file arrives
- Block caching is also common (NFS)
 - Typically integrated into shared block cache

Performance of Writes

- Writes at clients need to get to server(s) that store the data
 - And what about other clients caching that data?
- Not caching the writes is very expensive
 - Since they need to traverse the network
 - And probably be acknowledged
- Caching approaches improve performance at potential cost of consistency

Caching Writes For Distributed File Systems

- Write-back cache
 - Create the illusion of fast writes
 - Combine small writes into larger writes
 - Fewer, larger network and disk writes
 - Enable local read-after-write consistency
- Whole-file updates
 - No writes sent to server until *close(2)* or *fsync(2)*
 - Reduce many successive updates to final result
 - File might be deleted before it is written
 - Enable atomic updates, close-to-open consistency
 - But may lead to more potential problems of inconsistency

But potential
poor remote
consistency

Cost of Consistency

- Caching is essential in distributed systems
 - For both performance and scalability
- Caching is easy in a single-writer system
 - Force all writes to go through the cache
- Multi-writer distributed caching is hard
 - What do you do when one local cached copy is updated?
 - Must you know about other cached copies?
 - Will they see the update? When?
 - What if two local cached copies are updated?

Distributed Cache Consistency Approaches

- Time To Live
 - Delete items from cache after some time
 - Hoping you don't miss any writes meanwhile
- Check validity on use
 - Which requires remote access, defeating the purpose
- Only allow one writer at a time, per file
 - Too restrictive for most FS
- Change notifications
 - Notify cachiers when main copy gets an update
 - When exactly does the main copy hear about updates?

Usually the chosen solution

Scalability and Performance: Network Traffic

- Network messages are expensive
 - They use NIC and network capacity to carry them
 - And server CPU cycles to process them
 - Client delays awaiting responses
- Minimize messages/client/second
 - Cache results to eliminate requests entirely
 - Enable complex operations with single request
 - Buffer up large writes in write-back cache
 - Pre-fetch large reads into local cache

But remember

...

**If you don't send a message,
no one else knows it happened**

Conclusion

- Accessing data on remote machines is key to most distributed processing
- There are major challenges to doing so:
 - Performance
 - Consistency
 - Scalability
 - Reliability
- Solutions are available, but have associated costs and drawbacks
 - None of them are perfect for all purposes